

חיזוי פופולריות יחסית של כתבות חדשותיות באמצעות למידת מעבר מולטימודלית

מסמך לימודי מקיף - מלווה את פרויקט הגמר בקורס למידה עמוקה (Technion DS 26-529625)

מטרת המסמך הזו שונה מדוח פרויקט רגיל. הוא לא רק מציג מה עשינו ומה התוצאות - הוא נועד ללמד את החומר עצמו, כך שגם מי שלא מכיר את התחום יוכל להבין לעומק **מה כל מודל עושה, איך הוא עובד, ולמה בחרנו בו** ברגע הספציפי שבחרנו בו. כל מושג טכני מוסבר בפעם הראשונה שהוא מופיע, בשפה פשוטה ואינטואיטיבית, ורק אחר כך מיושם על הפרויקט הספציפי שלנו. אפשר לקרוא את המסמך מקצה לקצה כסיפור רציף אחד.

פרק 0: על מה בכלל מדובר, ולמה זה קשה

אתר חדשות כמו mako.co.il או n12.co.il מפרסם עשרות כתבות ביום. לפני שכתבה מתפרסמת, לעורך יש בעצם שני "נכסים" בלבד להחליט על סמכם אם היא תצליח: **הכותרת** שהוא כתב, ו**התמונה הראשית** (thumbnail) שהוא בחר. השאלה שהפרויקט הזה שואל היא פשוטה לניסוח אבל לא טריוויאלית לפתרון:

אם ניתן למודל רק את הכותרת, את התמונה, ואת שם האתר - עוד לפני שהכתבה התפרסמה ולפני שיש לה אפילו צפייה אחת - האם הוא יכול לנחש אם היא הולכת להיות מהצלחות הגדולות של החודש?

זו בעיה של **סיווג בינארי (Binary Classification)** - סוג הבעיה הבסיסי ביותר בלמידת מכונה, שבו המשימה היא להחזיר תשובה מתוך שתי אפשרויות בלבד: "כן", זו תהיה כתבה מצליחה" או "לא". בפועל המודל לא מחזיר "כן/לא" נוקשה אלא **הסתברות** - מספר בין 0 ל-1 שמבטא כמה הוא בטוח שהתשובה היא "כן". ככל שההסתברות גבוהה יותר, המודל "חושב" שהכתבה הזו תהיה מוצלחת יותר. איך בדיוק מודדים אם הניחוש הזה טוב - זו שאלה שנחזור אליה בהרחבה, כי היא מרכזית לכל הפרויקט.

מה זה אומר "כתבה מצליחה"?

ההגדרה הכי נאיבית לכתבה "מצליחה" היא מספר צפיות מוחלט - למשל "יותר מ-50,000 צפיות ב-48 השעות הראשונות". הבעיה: mako ו-n12 מקבלים כמויות תנועה שונות מאוד זו מזו, והתנועה גם משתנה מאוד לפי החודש (חדשות בוערות, עונתיות, אירועים). סף קבוע היה בעצם מודד בעיקר באיזה אתר ובאיזה חודש התפרסמה הכתבה, ולא באמת את איכות הכתבה עצמה.

לכן הגדרנו הצלחה **יחסית**: כתבה מסומנת כ"מצליחה" (1) אם היא נמצאת ב-**10% העליונים** של הכתבות **באותו אתר, באותו חודש קלנדרי**, לפי מספר הצפיות ב-48 השעות הראשונות. ההגדרה הזו נקראת לפעמים "peer group normalization" - אנחנו לא שואלים "האם הכתבה הזו הצליחה בעולם", אלא "האם היא הצליחה ביחס למתחרות הישירות שלה באותו הקשר". זו החלטת עיצוב מהותית שמשפיעה על כל מה שבא אחריה.

תוצאת ההגדרה: בכל פילוח של הדאטה (Train/Validation/Test – נסביר מיד מה זה) בדיוק כ-10% מהשורות הן "1" (מצליחות) ו-90% הן "0". זה נקרא **חוסר איזון מחלקות (Class Imbalance)**, וכפי שנראה בהמשך, יש לו השלכות עמוקות על איך בכלל מודדים הצלחה של מודל.

פרק 1: הדאטה – מאיפה הוא מגיע, ואיך מחלקים אותו

הדאטה מגיע ממחסן הנתונים הפנימי של "קשת" (Snowflake) – טבלת אירועי צפייה של mako.co.il ו-n12.co.il. עבור כל כתבה חילצנו: את הכותרת, את כתובת (URL) תמונת הפתיח, את שם האתר, את זמן הפרסום, ואת מספר הצפיות ב-48 השעות הראשונות (המשמש **רק** לבניית התווית/Label – המודל לעולם לא "רואה" את מספר הצפיות כקלט, אחרת היינו "מדליפים" למודל את התשובה עצמה).

לאחר עיבוד וסינון, נותרו **97,309 שורות**, החל מינואר 2024.

למה מחלקים לפי זמן ולא באקראי?

ברוב הקורסים הראשונים בלמידת מכונה מלמדים לחלק דאטה ל-train/validation/test. **באקראי**. בפרויקט הזה זו הייתה טעות אפשרית שנמנענו ממנה במודע: אם מחלקים באקראי, כתבות שקשורות זו לזו (למשל שתי כתבות על אותו אירוע חדשותי שהתפרסמו בהפרש של שעה) עלולות להיות אחת ב-train ואחת ב-test. המודל היה יכול "להעתיק" תבניות מהאחת לשנייה בלי באמת ללמוד שום דבר כללי – תופעה שנקראת **דליפת מידע (Data Leakage)**. בנוסף, השימוש המיועד למודל הוא תמיד לחזות **כתבות עתידיות** על סמך מה שנלמד מהעבר – לכן ההערכה חייבת לחקות בדיוק את זה.

הפתרון: **חלוקה כרונולוגית**. כל מה שקרה לפני תאריך מסוים משמש ל-**Training** (אימון המודל), החודש שאחריו משמש ל-**Validation** (בדיקות ביניים תוך כדי הפיתוח), והחודש שאחריו בכלל משמש ל-**Test** – ונוגעים בו **פעם אחת בלבד**, בסוף הפרויקט. נסביר בפרק 8 למה ההפרדה בין Validation ל-Test כל כך קריטית – זה למעשה אחד הלקחים המרכזיים של כל הפרויקט.

חלק	תקופה	מספר שורות	אחוז חיוביות
Train (אימון)	ינואר 2024 - אפריל 2026	87,334	כ-10%
Validation (בדיקות ביניים)	מאי 2026	3,722	כ-10%
Test (הערכה סופית, פעם אחת)	יוני 2026	3,745	כ-10%

תקלת איכות תמונה שגילינו תוך כדי

כתובת התמונה כפי שהיא נשמרת במסד הנתונים (`pic_furl`) מובילה כברירת מחדל **לתמונה זעירה בת 62x82 פיקסלים בלבד** – לא לתמונה המקורית. גילינו שהסרת סיומת מסוימת מה-URL מחזירה את התמונה המקורית במלוא הרזולוציה בכ-61% מהמקרים, עם גרסת גיבוי בגודל 175x232 פיקסלים שכמעט תמיד קיימת כרשת ביטחון. זו דוגמה טובה לכך שבפרויקטים אמיתיים חלק גדול מהעבודה הוא **איכות דאטה**, לא רק בניית מודלים – ולפעמים מגלים בעיות כאלה רק תוך כדי עבודה, לא בשלב התכנון.

החלטה: לא להשתמש בתגיות

לכתבות באתר יש גם "תגיות" (tags) נלוות. בדקנו וגילינו ש-49.5% מהכתבות בכלל לא מתויגות - כמעט חצי מהדאטה. לכן החלטנו מראש להוציא את התגיות מהמודל המרכזי לגמרי: **המודל הליבה רואה רק כותרת + תמונה + אתר**. זו החלטת עיצוב מכוונת, לא מגבלה טכנית - הוספת קלט שחסר בכמעט חצי מהמקרים הייתה רק מוסיפה מורכבות בלי תועלת ברורה.

פרק 2: איך בכלל מודדים אם מודל "טוב"? (וזו לא שאלה טריוויאלית)

לפני שבונים מודל כלשהו, חייבים להגדיר איך שופטים אותו - ובבעיה עם חוסר איזון מחלקות (90%/10%) זו שאלה עדינה יותר משנדמה.

למה "דיוק" (Accuracy) הוא מדד מטעה כאן

תארו לעצמכם מודל טיפשי שפשוט **תמיד** מנחש "לא", הכתבה לא תצליח". המודל הזה יהיה צודק ב-90% מהמקרים (כי 90% מהכתבות באמת לא ב-top 10%), ויקבל ציון **Accuracy של 90%** - נשמע מרשים, אבל המודל הזה חסר תועלת מוחלטת, כי הוא אף פעם לא מזהה אף כתבה מצליחה. זה הסיבה שבפרויקטים עם חוסר איזון מחלקות **אף פעם לא משתמשים ב-Accuracy כמדד מרכזי**.

ROC-AUC - האם המודל יודע לדרג נכון?"

ROC-AUC בודק שאלה שונה: אם ניקח באקראי כתבה מצליחה אחת וכתבה לא-מצליחה אחת, **מה הסיכוי שהמודל ייתן לכתבה המצליחה ציון (הסתברות) גבוה יותר?** מודל אקראי לגמרי מקבל 0.5 (כמו הטלת מטבע), מודל מושלם מקבל 1.0. זה מדד טוב ל"יכולת דירוג כללית" של המודל, אבל הוא **לא רגיש** לכך שיש הרבה יותר כתבות שליליות מחיוביות - הוא "ממוצע" על פני כל הזוגות האפשריים, כולל הרבה זוגות "קלים" (כתבה גרועה מובהקת מול כתבה טובה מובהקת).

PR-AUC - "מתוך מה שסימנתי כמצליח, כמה באמת הצליחו? וכמה מהמצליחים באמת תפסתי?"

זה המדד **העיקרי** שהשתמשנו בו בפרויקט הזה, כי הוא ישירות רלוונטי למצב האמיתי (10% חיוביים בלבד). הוא בנוי משני מרכיבים: - **Precision (דיוק)**: מתוך כל הכתבות שהמודל סימן כ"תהיה מצליחה", כמה מהן באמת הצליחו? - **Recall (כיסוי)**: מתוך כל הכתבות שבאמת הצליחו, כמה מהן המודל תפס?

יש כאן פשרה טבעית: אם המודל "זהיר" מאוד ומסמן רק כתבה אחת כמצליחה - ה-Precision שלו יכול להיות מושלם (100%) אבל ה-Recall יהיה כמעט אפסי (פספס כמעט את כולן). PR-AUC מודד את האיזון הזה על פני **כל** רמות הזהירות האפשריות של המודל, ונותן ציון יחיד. חשוב להבין: כאשר יש רק 10% חיוביים, PR-AUC של מודל אקראי לגמרי הוא **כ-0.10** (לא 0.5!) - לכן כל שיפור מעבר ל-0.10 הוא כבר סיגנל אמיתי, וההבדלים בין המודלים בפרויקט הזה (0.14 עד 0.24) הם משמעותיים גם אם הם נראים "קטנים" במבט ראשון.

Precision@K – המדד הכי “עסקי”

בפועל, השימוש המיועד למודל הוא **לדרג** רשימת כתבות מועמדות ולסמן את המבטיחות ביותר לעורך – לא לסווג כל כתבה בנפרד. **Precision@K** שואל: “אם ניקח את K הכתבות שהמודל הכי בטוח לגביהן (למשל K=50), כמה מהן באמת הצליחו?” זה המדד הכי קרוב לשימוש בפועל.

סיכום: השתמשנו בארבעה מדדים במקביל – ROC-AUC (יכולת דירוג כללית), PR-AUC (המדד המרכזי, בגלל חוסר האיזון), F1 (איזון יחיד בין Precision ל-Recall בנקודת סף אחת), ו-Precision@K (המדד הכי מעשי).

פרק 3: מודלי בסיס (Baselines) – למה בונים אותם קודם, לפני שבונים משהו “חכם”

לפני שבונים מודל מולטימודלי מורכב, בונים **ארבעה מודלי בסיס פשוטים**, שכל אחד רואה רק חלק מהמידע. המטרה היא לא רק “בדיקת ביצועים” – היא **לבודד** את התרומה של כל סוג קלט בנפרד, כדי שאחר כך נוכל לדעת אם המודל המורכב באמת משתמש נכון בכל המידע שיש לו.

3.1 TF-IDF + Logistic Regression (כותרת בלבד, ללא למידה עמוקה בכלל)

זה הבסיס הכי “קלאסי” – בלי רשתות נוירונים בכלל. **TF-IDF** (Term Frequency - Inverse Document) – הופך כל כותרת לוקטור מספרים: לכל מילה בשפה יש “משבצת”, והערך במשבצת גבוה אם המילה מופיעה הרבה בכותרת הזו אבל נדירה יחסית בכלל הכותרות (מילים כמו “כתבה” שמופיעות בכל מקום מקבלות משקל נמוך; מילים ייחודיות כמו “רעידת אדמה” מקבלות משקל גבוה). **Logistic Regression** הוא מודל סטטיסטי פשוט שלומד משקל לכל מילה כזו ומחבר אותם לניחוש סופי. המודל הזה קובע “רצפה” – האם יש בכלל סיגנל בכותרת, גם בלי שום טכנולוגיה מתוחכמת.

תוצאה: ROC-AUC 0.660, PR-AUC 0.208.

3.2 BERT עברי (כותרת בלבד, למידה עמוקה אמיתית)

BERT הוא סוג של **Transformer** – ארכיטקטורת רשת נוירונים ששינתה את כל תחום עיבוד השפה הטבעית (NLP) מאז 2018. ההבדל המהותי מהמודל הקודם: **BERT כבר אומן מראש** (Pretrained) על כמויות ענק של טקסט (ויקיפדיה, אתרי חדשות ועוד), ובתהליך הזה “למד” הבנה כללית מאוד עמוקה של שפה – דקדוק, הקשר, משמעות מילים לפי הסביבה שלהן. אנחנו לא מלמדים אותו שפה מאפס; אנחנו רק **מכוונים** (Fine-tuning) אותו למשימה הספציפית שלנו: לקחת את ההבנה הכללית שלו ולחבר אותה לניחוש “מצליח / לא מצליח”.

זה הרעיון המרכזי שנקרא **Transfer Learning (למידת מעבר)** – שם הקורס עצמו, ולב הפרויקט כולו: **להשתמש במודל שכבר יודע המון על העולם, ורק “להעביר” (transfer) את הידע הזה למשימה חדשה וספציפית, במקום ללמד הכל מאפס.**

השתמשנו ב-AlephBERT - גרסת BERT שאומנה במיוחד על טקסט עברי (בניגוד למודלים רב-לשוניים שרואים עברית רק כשבריר קטן מהדאטה שלהם). זה פרט שיתברר כקריטי בהמשך.

תוצאה: ROC-AUC 0.704, PR-AUC 0.212 - הבסיס הכי חזק מבין הארבעה, וכפי שנראה, מודל קשה מאוד לעקוף.

3.3 - תמונה בלבד

תמונת הפתיח מוזנת למודל ראייה מאומן-מראש (SigLIP2, נסביר עליו לעומק בפרק הבא), שממיר אותה לוקטור מספרים (**Embedding**) שמייצג את "המשמעות החזותית" שלה, ומעליו מודל Logistic Regression פשוט. **תוצאה: ROC-AUC 0.619, PR-AUC 0.148.** חלש בהרבה מכותרת בלבד - לא מפתיע: תמונת פתיח לרוב פחות אינפורמטיבית מכותרת שנכתבה במיוחד כדי "למכור" את הכתבה.

3.4 - חיבור נאיבי של כותרת ותמונה (Naive Concatenation) - התוצאה הכי חשובה מכל הבסיסים

כאן לוקחים את שני ה-Embeddings (של הטקסט ושל התמונה) - **בלי שום אימון משותף נוסף** - ומדביקים אותם זה לצד זה (Concatenation) לוקטור אחד ארוך יותר, ומזינים ל-Logistic Regression אחד.

תוצאה: ROC-AUC 0.619, PR-AUC 0.147 - כמעט זהה לתמונה-בלבד, וגרוע יותר מכותרת-בלבד!

זו תוצאה חשובה ומפתיעה: **הדבקה נאיבית של שני מקורות מידע לא רק שלא עוזרת - היא פוגעת.** הסיגנל החזק (טקסט) "מדולל" על ידי הסיגנל החלש (תמונה), כי אין שום מנגנון שמלמד את המודל איך לשקלל נכון בין השניים. זו ההצדקה המרכזית לכל מה שבא בהמשך: **כדי שתמונה וטקסט יתחברו לכדי משהו טוב יותר משניהם בנפרד, הם חייבים להיות מאומנים יחד (jointly), לא רק "מודבקים" בסוף.**

מודל	קלט	ROC-AUC	PR-AUC
TF-IDF + Logistic Regression	כותרת	0.660	0.208
BERT עברי (מכוון)	כותרת	0.704	0.212
תמונה בלבד	תמונה	0.619	0.148
חיבור נאיבי	כותרת+תמונה	0.619	0.147

פרק 4: המודל המרכזי הראשון - SigLIP2, ואימון בשלבים

מהו מודל Vision-Language, ומה זה בכלל אומר "מרחב משותף"?

SigLIP2 הוא מודל מסוג **Vision-Language** - מודל שמבין גם טקסט וגם תמונות, **באותו מרחב וקטורי**. הרעיון: בזמן האימון המקורי שלו (על ידי הצוות שיצר אותו, לפני שהגיע אלינו), הוא ראה מאות מיליוני זוגות של (תמונה, כיתוב-טקסט-שמתאר-אותה) מהאינטרנט, ולמד להמיר כל תמונה וכל משפט לנקודה במרחב רב-

ממדי - כך שתמונה וטקסט שמתארים את אותו הדבר יהיו קרובים זה לזה במרחב הזה, ותמונה וטקסט לא-קשורים יהיו רחוקים. תהליך האימון הזה נקרא **Contrastive Learning** - בכל צעד המודל רואה תמונה אחת מול הרבה כיתובים (רק אחד נכון), ולומד למשוך את הכיתוב הנכון קרוב ולדחוף את השאר רחוק.

התוצאה: SigLIP2 "מגיע" אלינו כבר עם **הבנה מובנית** של הקשר בין תמונות למילים - בדיוק סוג ההבנה שאנחנו צריכים בשביל "האם התמונה הזו ותומכת בכותרת הזו יוצרות יחד כתבה מוצלחת".

בדיקה מקדימה (לפני שהתחלנו לבנות משהו): בדקנו אם SigLIP2 בכלל "מבין" עברית (הוא אומן בעיקר על אנגלית, ועברית היא רק שבריר קטן מהדאטה שלו) - לקחנו כ-950 זוגות כותרת-תמונה אמיתיים, ובדקנו האם המודל (בלי שום אימון נוסף מצידנו) מצליח לשייך נכון כותרת לתמונה שלה מתוך כל התמונות במדגם. התוצאה: דיוק גבוה פי כ-28 מניחוש אקראי - סימן חיובי משמעותי שיש שם הבנה אמיתית, שהצדיק להמשיך.

הבעיה של "כמה לגעת" במודל מאומן-מראש

זו אחת השאלות המרכזיות בכל פרויקט Transfer Learning: המודל המאומן-מראש (375.6 מיליון פרמטרים!) כבר "יודע" המון. השאלה היא **כמה חופש לתת לו לזוז** בזמן ההתאמה שלנו למשימה הספציפית - ויש כאן מתח בסיסי:

- **יותר מדי נוקשות** (לא מזיזים כלום) → המודל לא מצליח להתאים את עצמו למשימה הספציפית שלנו, ונשאר "כללי מדי".

- **יותר מדי חופש** (מזיזים הכל) → על דאטה מוגבל (87 אלף שורות, לא מיליארדי דוגמאות כמו באימון המקורי), המודל עלול פשוט **לשנן** את דוגמאות האימון במקום ללמוד תבנית כללית. זו התופעה שנקראת **Overfitting (התאמת יתר)** - אחד המושגים החשובים ביותר בלמידת מכונה.

אנלוגיה ל-Overfitting: תלמיד שמתכוון למבחן על ידי שינון התשובות המדויקות של מבחני העבר, בלי להבין את החומר - הוא יצליח מצוין אם יקבל בדיוק אותן שאלות שוב, אבל ייכשל לגמרי במבחן חדש עם שאלות דומות אך לא זהות. מודל שעושה Overfitting "משנן" את דוגמאות האימון (Training Accuracy גבוה מאוד), אבל מתפקד גרוע על דאטה חדש שלא ראה (Validation נמוך) - בדיוק כמו התלמיד.

בגלל המתח הזה, לא ניגשים לבעיה בבת אחת - עושים את זה **בשלבים**, מהזהיר ביותר למסוכן ביותר:

שלב אי (Stage A) - Backbone קפוא לגמרי

כאן "נועלים" (Freeze) את כל 375.6 מיליון הפרמטרים של SigLIP2 במקומם - הם לא זזים בכלל בזמן האימון. מוסיפים רק **"ראש" (Head)** קטן חדש בסופו - רשת קטנה שמקבלת את ה-Embeddings שכבר SigLIP2 חישב, ולומדת לתרגם אותם לניחוש "מצליח/לא". רק ה-Head הזה, **395,793 פרמטרים בלבד (0.11% מהמודל!)**, בכלל מתעדכן בזמן האימון.

תוצאה: ROC-AUC 0.651, PR-AUC 0.167. שיפור אמיתי מעל הבסיס הנאיבי (0.619→0.651), אבל עדיין לא עוקף את BERT הטקסט-בלבד (0.704). זה לא מפתיע - Backbone קפוא לגמרי הוא הכי "בטוח" אבל גם הכי מוגבל ביכולת הסתגלות למשימה שלנו.

שלב ב' (Stage B) – שחרור 4 השכבות העליונות

הפעם משחררים (Unfreeze) את 4 השכבות העליונות של הרשת ומאפשרים להן להתעדכן – **57.1 מיליון פרמטרים (15.2% מהמודל)**. כדי שהחלקים שכבר "יודעים המון" לא יזוזו מהר מדי בזמן שהחלק החדש (ה-Head) עדיין לומד מאפס, משתמשים בטכניקה שנקראת **Discriminative Learning Rate** – קצב למידה שונה לכל חלק: ה-Head לומד ב"מהירות" רגילה, אבל השכבות המשוחררות של ה-Backbone לומדות ב-**עשירית** מהמהירות, כדי לשמר בזהירות את מה שהן כבר יודעות.

תוצאה: כישלון מהיר. ביצועי ה-Validation הגיעו לשיא כבר ב-Epoch הראשון (מונח שנסביר: **Epoch** = מעבר שלם אחד על כל דוגמאות ה-Training) ואז **התדרדרו** בהתמדה – סימן קלאסי ל-Overfitting: יותר מדי פרמטרים חופשיים (57 מיליון) על פני יחסית מעט דוגמאות אימון (87 אלף) – המודל התחיל "לשנן" את סט האימון. עצרנו את האימון מוקדם (**Early Stopping** – נסביר גם את זה בהמשך) ולא עקפנו את שלב א'.

שלב ג' (Stage C) – LoRA: איך "לומדים המון" בלי "לזוז המון"

הפתרון לבעיית שלב ב' הוא טכניקה חכמה בשם **LoRA (Low-Rank Adaptation)**. הרעיון האינטואיטיבי: במקום לתת לרשת המקורית (מטריצות ענק של מיליוני מספרים) לזוז חופשי, "מקפאים" אותה כמו בשלב א', אבל **לצד** כל שכבה חשובה מוסיפים זוג מטריצות קטנות מאוד – "תיקון" זעיר שנלמד, שמתווסף לפלט המקורי. המטריצות הקטנות האלה הן **"Low-Rank"**: המידע שהן יכולות לייצג מוגבל בכוונה (חושבו כמו "רזולוציה נמוכה" של תיקון, במקום תיקון מלא וחופשי) – כך שיש הרבה פחות "מקום" בשביל לשנן דברים ספציפיים, אבל עדיין מספיק כדי ללמוד התאמה שימושית.

התוצאה המספרית: רק כ-**986,000 פרמטרים נלמדים (0.28% מהמודל)** – פחות מפי 58 משלב ב', אבל בכל זאת יותר מפי 2.5 משלב א'. ה"מתיקה" הזו – הרבה יותר קיבולת למידה משלב א', הרבה פחות מסוכנת משלב ב' – היא בדיוק הסיבה ש-LoRA כל כך פופולרי היום בתעשייה.

תוצאה: ROC-AUC 0.659, PR-AUC 0.184 – הטוב מבין שלושת שלבי SigLIP2, ויציב על פני 10 Epochs מלאים, בלי קריסה. עדיין לא עוקף את (0.704 / 0.212) BERT, אבל ה**יציבות** שלו (בניגוד לקריסה של שלב ב') מדגימה בדיוק את הרעיון של LoRA.

שלב	פרמטרים נלמדים	ROC-AUC	PR-AUC	תוצאה
א' – קפוא לגמרי	395.8K (0.11%)	0.651	0.167	משתפר מעל הבסיס, לא עוקף BERT
ב' – שחרור 4 שכבות	57.1M (15.2%)	–	–	Overfitting מהיר, נכשל
ג' – LoRA	986K (0.28%)	0.659	0.184	הטוב מבין השלושה, יציב

מסקנת ביניים: אף אחד משלושת השלבים לא עקף את BERT הטקסט-בלבד. זו לא הייתה סיבה לוותר – זו הייתה נקודת המוצא לשלב הבא: אם לא הקיבולת של המודל היא הבעיה, אולי **הדרך שבה מחברים טקסט לתמונה** (ה-Fusion) היא הבעיה.

פרק 5: מנגנון ה-Attention – לתת למודל “לשאול שאלות”

מהו Attention, אינטואיטיבית

עד עכשיו, איך בדיוק “מחברים” את ייצוג הטקסט לייצוג התמונה בתוך המודל? בכל השלבים למעלה, השתמשנו ב**חיבור פשוט (Concatenation)** – לוקחים את הווקטור המסכם של הטקסט ואת הווקטור המסכם של התמונה, ומדביקים אותם זה לצד זה. זה בדיוק כמו ה-Baseline הנאיבי מפרק 3.4, רק שעכשיו זה קורה בתוך מודל שכן מאומן יחד – אבל עדיין: כל צד “מדבר” בנפרד, ורק בסוף נפגשים.

Attention הוא מנגנון שמאפשר למודל לעשות משהו הרבה יותר עדין: **לתת לחלק אחד של הקלט “לשאול שאלה” על חלק אחר, ולקבל תשובה ממוקדת**. הרעיון (בצורה הכי אינטואיטיבית): במשפט “הכלב רדף אחרי החתול כי הוא היה רעב”, כדי להבין למי “הוא” מתייחס, קורא אנושי לא קורא מילה-מילה בבידוד – הוא “מסתכל אחורה” ובודק אילו מילים קודמות הכי רלוונטיות. Attention עושה בדיוק את זה באופן ממוחשב: לכל מילה יש “שאלתה” (Query – “מי אני מחפש?”), ולכל מילה אחרת יש “מפתח” (Key – “מה אני יכול להציע?”); המודל מחשב עד כמה כל שאלתה “מתאימה” לכל מפתח, ומשתמש בהתאמות האלה כדי למשקל את המידע הרלוונטי.

Cross-Attention: הטקסט “שואל” את התמונה

ביישום שלנו הוספנו **Cross-Attention** דו-כיווני: הייצוג המסומן של הטקסט “שואל שאלה” ישירות על 196 ה“טלאים” (Patches) הגולמיים של התמונה (SigLIP2 מחלק כל תמונה לרשת של טלאים קטנים ומייצג כל אחד בנפרד, לפני הסיכום הכולל) – כלומר, במקום להשוות “תמצית טקסט” מול “תמצית תמונה”, הטקסט יכול “להביט” ישירות בחלקים ספציפיים של התמונה ולהחליט אילו מהם רלוונטיים. באותו האופן, גם התמונה המצומצמת “שואלת” על טוקני הטקסט הגולמיים. זה עקרונית מנגנון עשיר בהרבה מהדבקה פשוטה.

הניסוי המדעי: לבודד משתנה אחד בכל פעם

זו נקודה חשובה מאוד מבחינה מתודולוגית, ששווה להתעכב עליה: הניסוי **הראשון** שערכנו (Model 1) שינה **כמה דברים בבת אחת** – גם עבר ל-Cross-Attention, גם שילב שחרור-שכבות ו-LoRA יחד (במקום כל אחד בנפרד כמו בשלבים ב' ו'ג'), וגם השתמש בדאטה מסונן (רק תמונות ברזולוציה מלאה, במקום כל הדאטה). **התוצאה הייתה גרועה יותר** משלב ג' – אבל לא היה אפשר לדעת **מה בדיוק** גרם לכך, כי שלושה משתנים השתנו יחד.

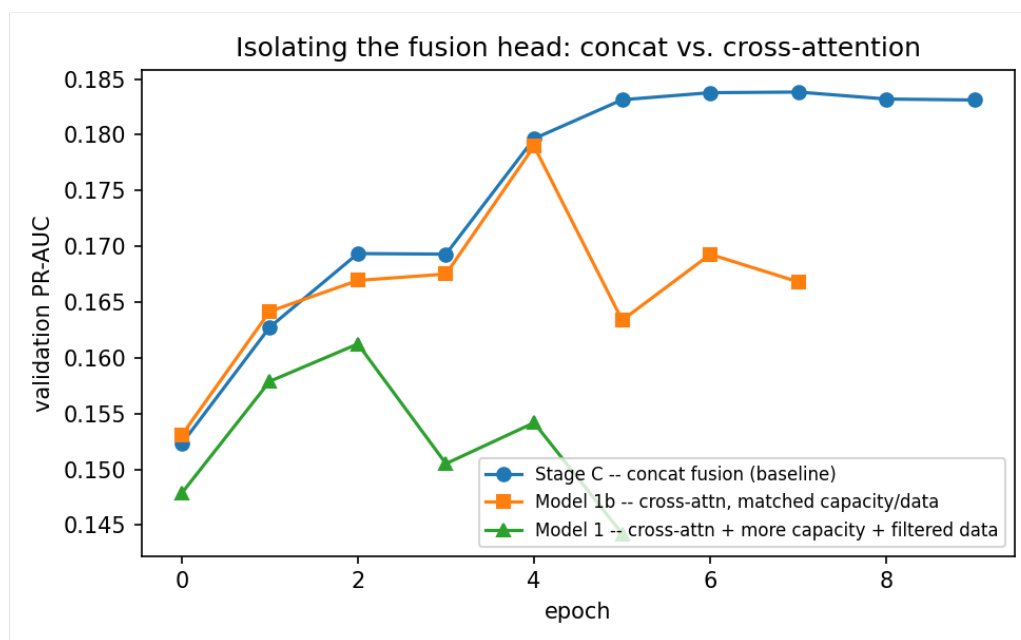
זה בדיוק העיקרון מאחורי **מחקר מבוקר (Controlled Experiment)** – ובלמידת מכונה קוראים לזה **Ablation Study**: כדי לדעת אם שינוי מסוים עוזר או מזיק, חייבים לשנות **רק אותו**, ולהשאיר את כל השאר קבוע. לכן ערכנו ניסוי שני, ממוקד (Model 1b): **בדיוק** אותה קיבולת כמו שלב ג' (LoRA בלבד, ללא שחרור

שכבות), **בדיוק** אותו דאטה (המלא, לא מסונן) - **המשתנה היחיד שהשתנה הוא סוג ה-Fusion**: Cross-Attention במקום Concatenation.

תוצאה: ROC-AUC 0.643, PR-AUC 0.179 - מעט מתחת לשלב ג' (0.184 / 0.659), באופן עקבי לאורך כל הריצה.****

ניסוי	Fusion	קיבולת	דאטה	ROC-AUC	PR-AUC
שלב ג'	Concatenation	LoRA בלבד	מלא	0.659	0.184
Model 1b (מבודד)	-Cross Attention	LoRA בלבד (זהה)	מלא (זהה)	0.643	0.179
Model 1 (לא מבודד)	-Cross Attention	שחרור+LoRA יחד	מסונן	0.625	0.161

מסקנה נקייה ומדעית: כשמבודדים את משתנה ה-Fusion בלבד - Cross-Attention (כפי שמומש כאן) **לא היה הפתרון וגם לא היה הבעיה** של Model 1. הקריסה של Model 1 נבעה בעיקר מהקיבולת הנוספת (שחרור שכבות + LoRA יחד) ומסינון הדאטה - לא ממנגנון החיבור עצמו. זה לקח חשוב: לפעמים "הפתרון היצירתי" שדמיינו לא בהכרח הבעיה האמיתית, וניסוי מבוקר נכון הוא הדרך היחידה לדעת בוודאות.



PR-AUC על פני Epochs: שלב ג' (Concatenation, בכחול) נשאר באופן עקבי מעל Model 1b (Cross-Attention) בקיבולת זהה, בכתום) - ההבדל קטן אך יציב. Model 1 (ירוק, קיבולת נוספת + דאטה מסונן) קורס אחרי Epoch 4.

פרק 6: כשהמודל הכללי לא מספיק – המודל ה-Hybrid

בשלב הזה כבר בדקנו את SigLIP2 בכל שילוב סביר (שלושה שלבי אימון, שני סוגי Fusion) – ואף אחד מהם לא עקף BERT הטקסט-בלבד. במקום להמשיך לדחוף באותו כיוון, עברנו לארכיטקטורה **שונה לגמרי**, שתוכננה כבר בתכנון המקורי כ"תוכנית גיבוי" למקרה שהעברית תהיה בעיה.

למה מודל אחר בכלל?

ההבדל המהותי: SigLIP2 הוא מודל **רב-לשוני** – הוא ראה בעיקר אנגלית באימון המקורי שלו, ועברית היא רק שבריר קטן. **AlephBERT**, לעומת זאת, אומן **מלכתחילה ורק** על טקסט עברי. ההשערה: אולי הבעיה האמיתית לא הייתה "כמה לגעת במודל" (מה שבדקנו בשלבים אי-ג') אלא **איזה מודל טקסט משתמשים בו בכלל**.

הארכיטקטורה: שני מודלים שמעולם לא "הכירו" זה את זה

זה הבדל מהותי נוסף מ-SigLIP2: **AlephBERT** (טקסט) ו-**ResNet50** (תמונה) הם שני מודלים שאומנו **בנפרד לגמרי**, כל אחד על סוג דאטה משלו, ומעולם לא ראו זה את זה לפני שאנחנו חיברנו אותם. זה שונה מ-SigLIP2, שבו הטקסט והתמונה **כן** אומנו יחד מלכתחילה במרחב משותף.

ResNet50 הוא רשת נירונים "קונבולוציונית" (CNN) – סוג הרשת הקלאסי לעיבוד תמונות, שסורק את התמונה בטלאים קטנים וקטנים יותר בכל שכבה, ולומד לזהות תבניות בהיררכיה עולה: קווים ופינות בשכבות הראשונות, צורות וטקסטורות באמצע, אובייקטים שלמים (פנים, בעלי חיים, רהיטים) בשכבות העמוקות. השם "ResNet" (Residual Network) מתייחס לחידוש המרכזי שלו – "קיצורי דרך" (Skip Connections) שמאפשרים לאותות לדלג מעל שכבות, מה שמאפשר לאמן רשתות עמוקות בהרבה מבלי שהאימון "יישבר" (בעיה טכנית שהייתה נפוצה לפני ResNet). המודל מאומן-מראש על מיליוני תמונות מ-ImageNet (מאגר תמונות כללי, לא חדשות) – כלומר הוא כבר "יודע לראות" באופן כללי, אבל לא ספציפית מה הופך תמונת חדשות למושכת.

איך מחברים אותם: מוציאים ייצוג מסוכם מכל אחד מהמודלים (טקסט: הטוקן הראשון "CLS" של AlephBERT, שנועד לתמצת את כל המשפט; תמונה: ה-Embedding הסופי של ResNet50 אחרי הסרת שכבת הסיווג המקורית שלו), מוסיפים גם Embedding נלמד קטן לשם האתר (mako/n12), מדביקים הכל יחד (Concatenation – כן, שוב חיבור פשוט, לא Cross-Attention, כי כאן אין אפילו מרחב Embedding משותף מלכתחילה כמו ב-SigLIP2, כך שאין סיבה חזקה לצפות שתשומת-לב מורכבת תעזור יותר – ובאמת, פרק 5 כבר הראה שזה לא היה הפתרון), ומזינים ל-MLP (רשת קטנה נוספת) שמפיקה ניחוש סופי.

מסע הכיוון: שלושה ניסיונות, שלושה לקחים

גרסה ראשונה (v1): משחררים שכבה עליונה אחת של Block + AlephBERT אחרון (layer4) של ResNet50. **תוצאה: שיא ב-Epoch הראשון (PR-AUC 0.212, ROC-AUC 0.660) – בדיוק תיקו עם BERT (!)** ואז **ירידה עקבית** – שוב Overfitting, אבל הפעם בקיצוניות פחותה מ-Stage B. זו הייתה התוצאה המולטימודלית הכי טובה עד כה, אבל עדיין לא **עוקפת** את BERT.

ניסיון כיוונון ראשון – באג שתפסנו והבנו: בניסיון לתפוס את השיא המוקדם יותר בדיוק, שינינו את תדירות בדיקות ה-Validation **מפעם באפוף לכל 500 צעדים**. אבל השארנו בטעות את פרמטר ה-**Early Stopping** **Patience** (כמה בדיקות רצופות בלי שיפור לפני שעוצרים) ללא שינוי, על הערך 3. הבעיה: 3 בדיקות כל 500 צעדים = **1,500 צעדים** של סבלנות בלבד – בעוד שהכוונה המקורית (3 בדיקות כל Epoch שלם) הייתה בעצם **כ-8,190 צעדים**. פי 5.5 פחות סבלנות מהמתוכנן! התוצאה: האימון נעצר ב-Step 4000, **בדיוק כש-ROC** **AUC הגיע לשיא חדש בפרויקט (0.670) ועדיין נראה במגמת עלייה** – כמעט בוודאות זו הייתה רק **רעש סטטיסטי רגיל** בסט Validation קטן יחסית (3,722 שורות), לא Overfitting אמיתי. זה לקח הנדסי חשוב: כשמשנים פרמטר אחד (תדירות בדיקה), צריך לזכור לכייל פרמטרים תלויים אחרים (סבלנות ל-**Early Stopping**) יחד איתו.

הגרסה המתוקנת (exp07) – פריצת הדרך של הפרויקט: תיקנו את הסבלנות ל-16 בדיקות (כ-8,000 צעדים, שומר על תקציב הסבלנות המקורי אך שומר גם על בדיקות עדינות ותכופות). בנוסף, החלטנו על שינויים נוספים בכיוון "זהירות": **הקפאה מלאה** של ResNet50 (בלי שחרור layer4 בכלל – הקטע ה"גדול" יותר מבין שני החלקים המשוחררים ב-v1, ולכן החשוד המרכזי לגרימת ה-Overfitting), והעלאת **Weight Decay** מ-0.01 ל-0.05.

מהו Weight Decay? זהו סוג של **Regularization (רגולריזציה)** – עונש מתמטי קטן שמתווסף לפונקציית האימון, ומעודד את המודל להעדיף פתרונות עם משקלים (weights) קטנים ו"פשטניים" יותר, במקום להסתמך חזק מדי על תבנית ספציפית אחת בדאטה. אינטואיטיבית: זה כמו לומר למודל "אל תהיה בטוח מדי בשום דבר יחיד" – מה שמקשה עליו לשנן פרטים ספציפיים של דוגמאות האימון, ומעודד אותו ללמוד תבניות כלליות יותר שמתאימות גם לדאטה חדש.

תוצאה: הריצה השלימה את כל 10 ה-Epochs, בלי Early Stopping בכלל – יציבה לגמרי, בלי שום קריסה. שיא ב-Step 21,500.

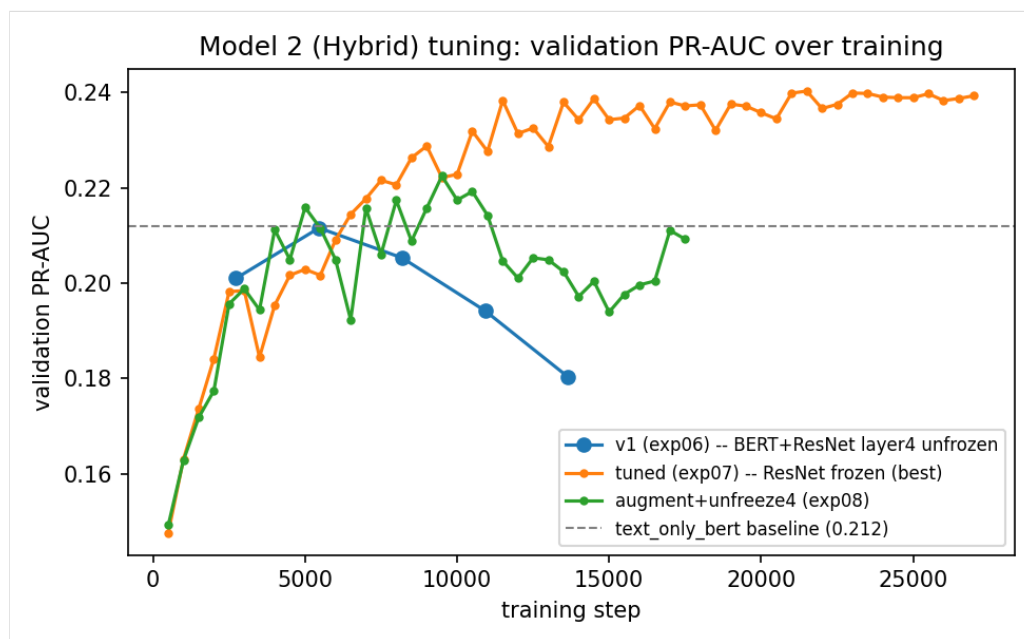
מודל	ROC-AUC	PR-AUC
BERT עברי (טקסט בלבד)	0.704	0.212
Model 2 (Hybrid, מכונן – exp07)	0.693	0.240
שלב ג' SigLIP2 (LoRA)	0.659	0.184
Model 2 v1 (לא מכונן)	0.660	0.212

זו הפעם הראשונה והיחידה בכל הפרויקט שמודל מולטימודלי עוקף את הבסיס הטקסטואלי החזק ביותר. שני שינויים היו מכריעים: (1) שימוש במודל טקסט עברי-ייעודי (AlephBERT) במקום מודל רב-לשוני (SigLIP2), ו-(2) רגולריזציה זהירה – הקפאת ResNet לגמרי, **Weight Decay** גבוה יותר, וסבלנות מכוילת נכון לתדירות הבדיקה.

ניסיון נוסף (exp08) – לבדוק אם אפשר לדחוף עוד יותר: אחרי ההצלחה, ניסינו לשחזר את הקיבולת הנוספת של layer4 (שעזרה ל-v1 להגיע לשיא-ROC-AUC גבוה), הפעם עם **הגברת נתונים (Data)**

Augmentation – טכניקה שמייצרת וריאציות אקראיות קלות של כל תמונת אימון (חיתוך אקראי, היפוך אופקי, שינוי צבע קל) כדי “להקשות” מעט על המודל וכך למנוע ממנו לשנן פיקסלים ספציפיים. חשוב לציין: הגברת נתונים כזו רלוונטית רק כשחלק מרשת התמונה בכלל מתאמן – בהקפאה מלאה (כמו ב-exp07) היא לא הייתה עושה שום דבר, כי אין מה “להקשות” על רכיב שלא לומד ממילא.

תוצאה: ROC-AUC 0.674, PR-AUC 0.222 – שיפור על פני v1 (מאשר שהגברת הנתונים באמת עזרה למנוע חזרה על אותה קריסה), אבל לא עוקף את exp07. מסקנה ברורה: הקיבולת הנוספת של layer4, גם עם רגולריזציה ישירה, פשוט לא משתלמת כאן – הקונפיגורציה הטובה ביותר נשארת exp07, עם ResNet קפוא לגמרי.



מסע הכיוון של Model 2 לאורך צעדי האימון: v1 (כחול) משתפר מהר ואז קורס; הגרסה המכוונת exp07 (כתום) מטפסת בהתמדה מעל קו הבסיס הטקסטואלי בלי שום קריסה; exp08 עם הגברת נתונים (ירוק) יציבה יותר מ-v1 אך לא מגיעה לרמת exp07.

פרק 7: טבלת ה-Ablation הסופית – התשובה המסכמת לשאלת המחקר

Ablation Study (שכבר הזכרנו בפרק 5) היא בעצם הרעיון של להסתכל על כל השילובים האפשריים של קלטים, אחד מול השני, כדי לבודד את התרומה ה**הנקייה** של כל מקור מידע. הרכבנו את הטבלה הסופית מהתוצאות שכבר היו קיימות (לא אימנו מודלים חדשים – כל אחד מה-Baselines כבר בודד בדיוק את שילוב הקלט הרלוונטי):

רכיב באבלציה	קלט	ROC-AUC	PR-AUC
כותרת בלבד	כותרת	0.660	0.208
כותרת (+תגיות)*	כותרת	0.704	0.212
תמונה בלבד	תמונה	0.619	0.148
כותרת+תמונה (הדבקה נאיבית)	כותרת+תמונה	0.619	0.147
המודל המלא (exp07)	כותרת+תמונה+אתר	0.693	0.240

*לתשומת לב: אף מודל בפרויקט לא קיבל תגיות בפועל כקלט (ראו פרק 1) - השורה הזו היא בעצם רק BERT טקסט-בלבד, כלול כאן כדי לשמור על צורת הטבלה המקובלת בתחום, לא כדי להטעות.

זו התשובה הישירה לשאלת המחקר של כל הפרויקט: המעבר מ"כותרת+תמונה מודבקים בצורה נאיבית" (PR-AUC 0.147) ל"מודל מלא, מאומן במשותף" (PR-AUC 0.240) הוא שיפור גדול - אבל הוא **לא** מיוחס רק לתמונה; הוא גם משקף מעבר ממודל טקסט רב-לשוני קפוא (SigLIP2) למודל טקסט עברי-ייעודי ומכוון (AlephBERT). ההשוואה הנקייה ביותר ל"האם התמונה עוזרת" היא מול הבסיס הטקסטואלי החזק ביותר (BERT): **0.240 מול 0.212**.

פרק 8: הרגע האמיתי - סט הבדיקה (Test Set), ולמה זה השיעור הכי חשוב בכל הפרויקט

למה לא סתם משתמשים ב-Validation כתשובה הסופית?

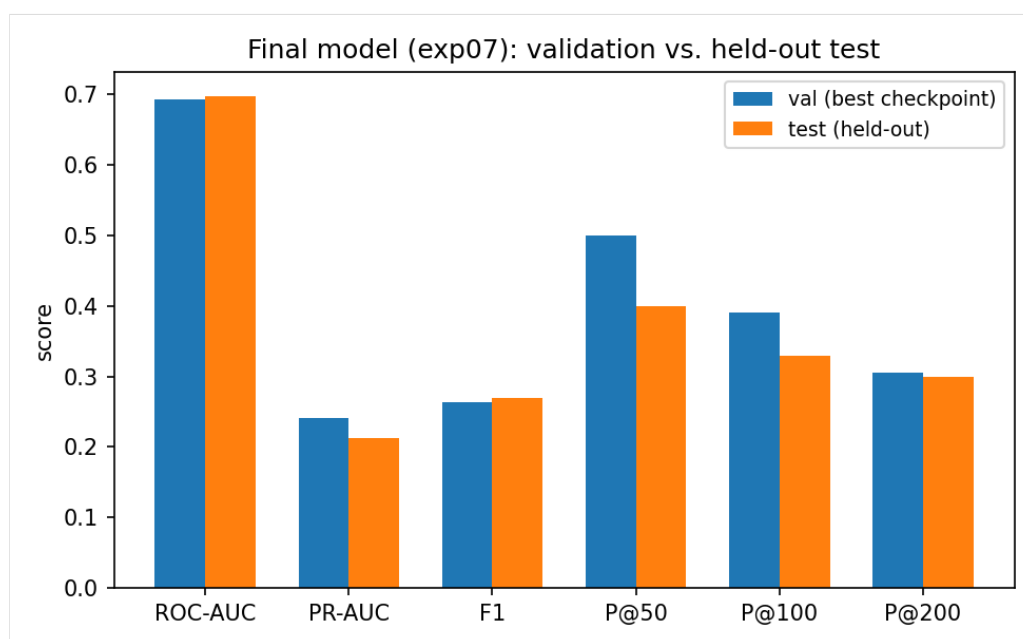
זו נקודה שקל לפספס אבל היא **קריטית** להבנת למידת מכונה בצורה נכונה. לאורך כל הפרויקט, כל החלטה - איזה שלב SigLIP2 הכי טוב, איזו קונפיגורציה של ה-Hybrid לשמור, מתי לעצור אימון - התקבלה על סמך ביצועים על סט ה-Validation. וזו בדיוק הבעיה: **אם עושים הרבה מאוד בדיקות והחלטות כנגד אותו סט Validation, יש נטייה שיטתית "להתמזל" עליו** - לא בכוונה, אלא סטטיסטית: מתוך כמה ניסיונות (exp06, exp07, exp08 ועוד), הניסיון שנראה הכי טוב על ה-Validation עשוי להיות טוב חלקית בזכות **רעש אקראי** של אותו סט ספציפי (3,722 שורות זה לא הרבה), לא רק בזכות שיפור אמיתי וכללי.

זה הסיבה שיש **שלוש** קבוצות דאטה, לא שתיים: ה-**Test Set** נשמר **בלי לגעת בו בכלל** לאורך כל הפרויקט, ומשמש **פעם אחת בלבד**, בסוף הסוף, אחרי שכל ההחלטות כבר ננעלו - כדי לקבל תמונה **הוגנת ובלתי-מוטה** של ביצועי המודל על דאטה שהוא, ואנחנו, מעולם לא ראינו.

התוצאות בפועל

קבוצה	ROC-AUC	PR-AUC	F1	P@50	P@100	P@200
Validation (ה-Checkpoint שנבחר)	0.693	0.240	0.263	0.50	0.39	0.305
Test (חדש) לגמרי, שימוש ראשון	0.697	0.212	0.270	0.40	0.33	0.30

ROC-AUC דווקא השתפר מעט (0.693→0.697) – סימן שיכולת הדירוג הכללית של המודל לא “השתננה” (Overfit) לסט ה-Validation באופן קלאסי. אבל **PR-AUC ירד באופן ברור (0.240→0.212)**, וגם **Precision@50 ירדו** – כלומר **היתרון שראינו מול BERT (0.240 מול 0.212 על Validation) כמעט ונעלם על ה-Test**: על ה-Test, ה-PR-AUC של המודל ההיברידי (0.212) כמעט זהה בדיוק ל-PR-AUC של BERT על Validation (0.212).



המודל הסופי: Validation מול Test על פני כל המדדים. ROC-AUC ו-F1 נשמרים (F1 אף משתפר מעט), בעוד PR-AUC – Precision@K יורדים לכיוון רמת הבסיס הטקסטואלי.

למה זה קרה – ואיך לחשוב על זה נכון

זו לא “כישלון” של המודל במובן הרגיל – זו תוצאה **צפויה** של תהליך העבודה עצמו: עברנו הרבה סבבי כיוון (exp06→exp07→exp08), וכל החלטה על איזה checkpoint או קונפיגורציה הכי טובים התבססה כל פעם על אותו סט Validation. זה בדיוק סוג ה”דליפה העדינה” שסט Test נועד לחשוף – לא דליפת מידע ישירה (המודל לא “ראה” את ה-Validation בתהליך האימון עצמו), אלא **דליפה דרך תהליך הבחירה** (Selection)

(Bias): כשבחרים את "המנצח" מתוך כמה ניסיונות לפי אותו מדד, המנצח הנבחר נוטה להיות מוטה כלפי מעלה ביחס לביצועים ה"אמיתיים" שלו, כי חלק מהניצחון שלו הוא מזל סטטיסטי ולא שיפור אמיתי וכללי.

זה למעשה אחד השיעורים הכי חשובים בכל תחום הדאטה סיינס: מדד ה-Validation, אחרי הרבה סבבי אופטימיזציה כנגדו, מפסיק להיות אומדן נטול-הטיה (Unbiased Estimate) לביצועי המודל האמיתיים. רק דאטה שממש לא נגעו בו במהלך תהליך קבלת ההחלטות נותן תשובה כנה.

מסקנה כנה: התשובה לשאלת המחקר נשארת **חיובית אך מתונה יותר** ממה שנראה באמצע הפרויקט: יש אינדיקציה אמיתית שהתמונה יכולה לתרום מעבר לכותרת (המודל ההיברידי היה היחיד שעקף את BERT על Validation, ועל Test הוא לפחות שקול לו ולא נופל ממנו) – אבל התועלת לא כה חד-משמעית כפי שנדמתה באמצע תהליך הכיוון.

פרק 9: מסקנות סופיות

האם התמונה עוזרת מעבר לכותרת? התשובה הכנה: **במידה מסוימת, אך לא בצורה חד-משמעית.** על Validation, לאורך הרבה סבבי כיוון, המודל המולטימודלי הטוב ביותר עקף את הבסיס הטקסטואלי החזק ביותר בפער נאה. על חודש בדיקה יחיד ולא-נגוע, הפער הזה כמעט נעלם על המדד העיקרי (PR-AUC) בזמן שנשמר על המדד המשני (ROC-AUC). לא קוראים לזה "התמונה לא תורמת כלום" אלא "התרומה אמיתית אך צנועה, וכנראה הוצגה בצורה מוגזמת באמצע הפרויקט בגלל תהליך הבחירה".

למה הגישה המתוחכמת יותר (Cross-Attention) נכשלה? הניסוי המבודד בפרק 5 נתן תשובה די סופית: לא מנגנון ה-Fusion עצמו הזיק – זו הייתה הקיבולת הנוספת שנוספה יחד איתו והדאטה המצומצם יותר. גישה יסודית יותר הייתה בוחנת Cross-Attention בקיבולת **זרה** במבחן רחב יותר, או תשומת-לב על פני **כל** רצף הטוקנים ולא רק שאילתות מצומצמות – שני כיוונים שלא נבדקו בגלל מגבלת הזמן.

למה המודל ההיברידי הצליח דווקא הוא, ולא SigLIP2? שני גורמים שפעלו יחד: מודל טקסט עברי-ייעודי אמיתי (AlephBERT) במקום מודל רב-לשוני שנחשף לעברית רק חלקית, ורגולריזציה שמרנית שנתגלתה תוך כדי איטרציה (הקפאת התמונה לגמרי, Weight Decay גבוה יותר, Early Stopping מכויל נכון). זה מרמז שההתאמה של מודל הטקסט לשפה הספציפית הייתה חשובה יותר, למשימה ולשפה הזו, מהיתרון התיאורטי של מרחב Embedding משותף שכבר אומן מראש כמו ב-SigLIP2.

מגבלות המחקר: (1) פיצול יחיד ל-Validation/Test (חודש אחד לכל אחד) – לא ניתן להעריך בביטחון כמה מהפער בין המדדים הוא רעש חודשי גרידא. (2) תגיות מעולם לא נבדקו בפועל כתרומה אפשרית. (3) איכות תמונות משתנה מאוד ולא תוקנה מחוץ לניסוי המבוקר היחיד שהתייחס אליה. (4) Seed אקראי בודד לכל קונפיגורציה – אין אומדן שונות בין הרצות.

מה היה משתפר עם עוד זמן: לבדוק על כמה חודשי Test שונים, לא רק אחד; לבחון בפועל את תרומת התגיות במקום להשמיט אותן; להריץ כל קונפיגורציה בכמה Seeds כדי לדעת כמה מהתוצאה היא שונות רגילה; ולהיות ממושמצים יותר לגבי מספר הפעמים שמסתכלים על ה-Validation לפני נעילת מודל סופי – הפער בין Validation ל-Test שתועד בפרק 8 הוא ההדגמה הכי ישירה לכך שהמשמעת הזו חשובה.